



Time Series Event Detection Examples: Anomalies, Change Points, Motifs

Harbinger: The Event Detection Pipeline

Event detection in time series identifies critical moments where statistical behavior shifts. This includes anomalies, sudden changes (change-points), or recurring patterns (motifs).

Harbinger offers a unified `fit` → `detect` → `plot` interface. Users `fit()` models to their data, `detect()` events, and `har_plot()` visualizes outcomes. This standardization allows seamless swapping of various detectors, simplifying analysis. Built on the powerful DAL Toolbox, Harbinger ensures robust and scalable event detection.

Fit

Train model on time series data

Detect

Identify flagged events in a data frame

Plot

Visualize event detection results

Reference: Ogasawara, E. et al. "Event Detection in Time Series." Springer, 2025. doi:10.1007/978-3-031-75941-3

Standard Pipeline: `harbinger()`

End-to-end demonstrations of the standard event detection pipeline across diverse time series datasets: non-stationarity, global temperature (monthly/yearly), multivariate, and Seattle weather.

```
# Install (once only)
#install.packages("harbinger")

# Load packages
library(daltoolbox)
library(harbinger)

# Load example datasets
data(examples_harbinger)
model <- harbinger()

# Example: non-stationarity
dataset <- examples_harbinger$nonstationarity
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# Example: global temperature (monthly)
dataset <-
examples_harbinger$global_temperature_monthly
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)

# Example: global temperature (yearly)
dataset <-
examples_harbinger$global_temperature_yearly
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

Step-by-Step Breakdown

- **Install/load** — `harbinger` depends on `daltoolbox`
- **data()** — loads ready-to-use time series datasets
- **harbinger()** — instantiates the standard pipeline object
- **fit()** — fits the model to the time series; re-fit to any dataset without reinitializing the model object
- **detect()** — runs event detection on the fitted model
- **har_plot()** — visualizes the series, detections, and ground-truth events

Datasets covered:

- **Non-stationarity** — synthetic series with structural shifts
- **Global temperature (monthly)** — long-run climate record at monthly resolution
- **Global temperature (yearly)** — same series aggregated annually
- **Multidimensional** — multivariate time series with correlated channels
- **Seattle weather (weekly)** — regional climate data at weekly granularity
- **Seattle weather (daily)** — same source at daily resolution

Anomaly Detection in Time Series

An anomaly, or outlier, is an observation that deviates significantly from the expected behavior of a time series. Identifying these unusual patterns is crucial for system monitoring, fraud detection, and predictive maintenance.

Point Anomalies

Single, isolated data points that significantly diverge from the rest of the series.

Contextual Anomalies

Data points normal in value but abnormal given their specific context or surrounding data.

Collective Anomalies

A sequence of data points that, as a group, are anomalous, even if individual points appear normal.

Trend-Embedded Anomalies

Subtle shifts or deviations within an overarching trend, indicating a change in the underlying pattern.

Harbinger detects anomalies by modeling the expected behavior of the time series (e.g., through residuals or statistical thresholds) and flagging observations that exceed a defined deviation margin. The effectiveness of detection heavily depends on selecting a detector suitable for the specific anomaly pattern.

Reference: Chandola, V., Banerjee, A., Kumar, V. (2009). "Anomaly detection: A survey." ACM Computing Surveys, 41(3), 1–58.

Recognizing Anomaly Patterns in Time Series

Take a guided tour through the most common anomaly patterns — isolated spikes, contextual outliers, trend-embedded deviations, and multi-event sequences — all detected through Harbinger's unified event detection pipeline.

```
# Load anomaly datasets
data(examples_anomalies)
model <- harbinger()

# Simple anomalies: isolated spikes
dataset <- examples_anomalies$simple
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)

# Contextual anomalies
dataset <- examples_anomalies$contextual
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# Time series with trend + anomalies
dataset <- examples_anomalies$trend
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)

# Multiple anomalies
dataset <- examples_anomalies$multiple
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

Step-by-Step

- **simple** — isolated spikes, straightforward to detect visually
- **contextual** — anomaly depends on the local context of the time series
- **trend** — anomalies embedded in a series with an upward or downward trend
- **multiple** — several anomalous events distributed across the series

📌 **Key concept:** Different anomaly patterns manifest in distinct ways within residual magnitudes — always visualize before fitting to choose the right event detection strategy.

Sequences, Amplitude Shifts & Variance Regimes

Advanced detection patterns: repeated anomalous subsequences, train/test splits, growing amplitude, and volatile variance — each demanding a tailored detector.

```
# Repeated anomalous sequences
dataset <- examples_anomalies$sequence
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# Train/test split
dataset <- examples_anomalies$tt
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# Warped train/test split
dataset <- examples_anomalies$tt_warped
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# Increasing amplitude
dataset <-
examples_anomalies$increasing_amplitude
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# Volatile variance
dataset <- examples_anomalies$volatile
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

Step-by-step

- **sequence** — repeated anomalous subsequences (collective anomalies)
- **tt / tt_warped** — evaluates event detection behavior across train/test splits, with and without temporal distortion
- **increasing/decreasing_amplitude** — scale shifts that grow or shrink over time
- **volatile** — unstable variance indicative of a high-volatility regime change-point

❏ **Key concept:** Amplitude shifts and variance instability are regime-level patterns — distinct from pointwise anomalies in a time series and requiring detectors specifically designed for change-point and motif-based detection.

Identifying Structural Shifts in Time Series

Change-point detection focuses on identifying precise moments when the statistical properties of a time series—such as mean, variance, or trend—undergo a significant and permanent shift. This differs fundamentally from anomaly detection, as change-points signal a lasting alteration in the system's underlying distribution, not just a transient deviation.

AMOC (At Most One Change)

Optimized for detecting a **single change-point**, useful when a clear structural break is expected within the data.

BinSeg (Binary Segmentation)

An efficient method for identifying **multiple change-points** by recursively dividing the series at suspected break points.

PELT (Pruned Exact Linear Time)

Offers an **optimal solution for multiple change-points**, guaranteeing accuracy while maintaining computational efficiency.

GFT (Generalized Fluctuation Test)

Primarily used for detecting **regression-based structural changes**, assessing parameter stability over time.

Chow Test

A statistical test for detecting a **single change-point** in a linear regression model, comparing fit across sub-periods.

Selecting the right algorithm depends on the nature of the expected shifts: AMOC and Chow are ideal for single, distinct breaks, while BinSeg and PELT are robust for scenarios involving multiple change-points. The GFT is best suited for detecting evolving relationships in regression models.

Reference: Killick, R., Fearnhead, P., Eckley, I. A. (2012). "Optimal detection of changepoints with linear cost." JASA, 107(500), 1590–1598.

Exploring Change-Point Scenarios

Walks through representative change-point scenarios: simple structural breaks, multiple breaks, and variance/volatility shifts. Classical approaches (AMOC/BinSeg/PELT) or regression-based methods (GFT/Chow) are recommended depending on the case.

```
# Load change-point datasets
data(examples_changepoints)
model <- harbinger()

# Simple change-point
dataset <- examples_changepoints$simple
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
         detection, dataset$event)

# Sinusoidal pattern with regime change
dataset <- examples_changepoints$sinusoidal
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
         detection, dataset$event)
```

```
# Incremental trend shifts
dataset <- examples_changepoints$incremental
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
         detection, dataset$event)

# Abrupt level jump
dataset <- examples_changepoints$abrupt
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
         detection, dataset$event)
```

Step-by-Step

- **simple** — a single structural break in the time series
- **sinusoidal** — regime change within a periodic time series
- **incremental** — trend that shifts gradually over time
- **abrupt** — sudden level jump (object ts)

📌 **Key concept:** Change-points mark where the statistical structure of a time series fundamentally shifts — unlike point anomalies, they affect the entire subsequent distribution.

Volatility, Amplitude, and Complex Regimes

Building on simple structural breaks, these advanced change-point scenarios explore variance shifts, growing amplitude, and time series with multiple chained regimes — cases where single-breakpoint detectors fall short.

```
# Volatility change-point (variance)
dataset <- examples_changepoints$volatility
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)

# Increasing amplitude
dataset <-
examples_changepoints$increasing_amplitud
e
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# Complex multi-regime series
dataset <- examples_changepoints$complex
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

Step-by-step

- **volatility** — variance shifts abruptly at a specific change-point
- **increasing_amplitude** — the scale of the time series grows progressively over time
- **complex** — multiple regimes chained together within the same series
- The Defaulting to continuous warning is expected for ts objects

📌 **Key concept:** Multi-regime time series require event detection algorithms capable of identifying multiple change-points — PELT is strongly preferred over AMOC in these cases.

Unveiling Patterns: Motifs and Discords

In time series analysis, identifying recurring patterns and unusual events is crucial. A **motif** is a frequently recurring subsequence—the "pattern that repeats"—while a **discord** is its opposite: the most unusual and anomalous subsequence.

Motifs

Frequently recurring subsequences, indicating typical or expected behaviors.

Discords

The most unusual subsequences, often signaling anomalies or critical events.

Matrix Profile

State-of-the-art algorithm for robust motif and discord discovery via distance computation.

SAX

Symbolic Aggregate approXimation discretizes series for efficient pattern matching.

These techniques are vital for applications such as **ECG heartbeat detection**, **industrial fault detection**, and **activity recognition**.

References: Yeh, C.-C. M. et al. (2016). "Matrix Profile I." IEEE ICDM. And: Tavenard, R. et al. (2020). tsmp. The R Journal. doi:10.32614/RJ-2020-021

Discovering Motifs in Time Series

Explore motif discovery — recurring subsequences in time series data — using Harbinger's unified interface across synthetic and real-world ECG datasets (MIT-BIH). Matrix Profile and SAX are the primary detection methods.

```
# Load motif datasets
data(examples_motifs)
model <- harbinger()

# Simple synthetic dataset
dataset <- examples_motifs$simple
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

```
# ECG: MIT-BIH record 100
dataset <- examples_motifs$mitdb100
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)

# ECG: MIT-BIH record 102
dataset <- examples_motifs$mitdb102
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie,
          detection, dataset$event)
```

Step-by-step

- **examples_motifs** — includes a synthetic time series and real ECG records
- **mitdb100/102** — recordings from the MIT-BIH Arrhythmia Database
- **Motif** — a frequently recurring subsequence; **discord** — the most unusual subsequence
- Matrix Profile and SAX provide the underlying algorithms for motif discovery

📌 **Key concept:** Motifs reveal recurring patterns in a time series; discords expose subsequence-level anomalies — both are detected through the same fit → detect → plot interface.



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

